Dynamic Programming

Arihant Jain
Electrical and Computer Engineering
UC San Diego

***Abstract*—This project considers the problem of planning in a 2-dimensional space using the principles of dynamic programming. We attempt to find the shortest path from a start location to a goal location, which could be through a locked door which needs to be unlocked. Our methods will involve implementing a label-correcting solution for the dynamic programming problem, and have insights into choices made for the configuration space and control space.**

I. Introduction

The goal of this paper is to explore how to solve for the shortest path to a goal tile in a discreet 2-dimensional environment, with obstacles such as walls and locked doors, which need to be unlocked with a key. Unlike regular planning approaches, such as those using algorithms like Dijkstra's or A* search, we will be finding a solution using dynamic programming(DP), and reformulating the problem as a Markov Decision Process (MDP), and then into a Deterministic Shortest Path (DSP) problem. For this goal, the primary challenge involves formulating the state space, control space, and value function to be used as part of the DSP. Finally, the approach in this paper also improves implementation performance by using a Label-Correcting (LC) algorithm, to prioritize states that have a low cost-to-arrive.

DP is used in robotics to solve many kinds of planning problems, ranging from our case of a discrete grid, to complex Bellman equations. This is because having a well-defined value function along with transition function enables us to solve the problem backwards from the goal to the start state, and find the lowest value for each discrete timestep. We illustrate how this method is implemented, and showcase results on our dataset.

II. Problem Formulation

The known environments are all square, with one door, and a single key. For this problem, we can formulate it as an MDP as follows:

State Space: $X = \{row, col, dir, has_key, door_state\}$

Where $row \in [1, width]$, $col \in [1, height]$ are the position of the robot, $dir \in \{0, 1, 2, 3\}$ is the direction it faces, $has_key \in \{0, 1\}$ is if the robot is carrying the key, and $door_state \in \{0, 1\}^2$ is whether each of the two doors is open.

Control Space: $U = \{MF, TL, TR, PK, UD\}$

Where MF moves the robot forward in the direction it is facing if possible, TL and TR change the robot's direction by turning it left or right respectively, PK picks up a key in front of the robot, if it exists, and UD unlocks a door in front of the robot, if it exists.

Motion Model: this is a deterministic motion model:

$$f(x,u) = \begin{cases} (row + dir[0], col + dir[1], dir, has_key, door_open) \\ \quad \text{if } u = MF \text{ and in bounds} \\ (row, col, dir - 1, has_key, door_open) \\ \quad \text{if } u = TL \\ (row, col, dir + 1, has_key, door_open) \\ \quad \text{if } u = TR \\ (row, col, dir, 1, door_open) \\ \quad \text{if } u = PK \text{ if has_key == 0 and key in front} \\ (row, col, dir, has_key, update_door_state) \\ \quad \text{if } u = UD \text{ if door in front} \end{cases}$$

Initial state: $s = x_0 \in X$ is given by environment initialization.

Planning horizon T is infinite, since we want to find the shortest possible solution in any amount of time.

Stage cost:

$$l(u) = \begin{cases} 3 & \text{if } u = MF \\ 1 & \text{if } u = TL \\ 1 & \text{if } u = TR \\ 2 & \text{if } u = PK \\ 5 & \text{if } u = UD \end{cases}$$

Terminal cost:

$$q(x) = \begin{cases} \infty & \text{if } x[:2] \neq g \\ 0 & \text{if } x[:2] = g \end{cases}$$

where $g \in [1, 10]^2$ is the goal cell.

For the unknown environments, we had to reformulate our state space to include the potential key and goal positions:

State Space: $X = \{row, col, dir, has_key, door_state, key_pos, door_pos\}$

This expanded state space allows us to encode all potential goal and key positions, as well as all initial door states for 2 doors. We can then retain the same problem formulation as the known environments.

III. Methods and Approach

Given the above MDP formulation, we can now solve the optimal control problem using dynamic programming and find a optimal policy to minimize our value function. For this, we

solve the forward DSP problem, which is the exact same in our case as the backward problem. We use the Deterministic Shortest Path via Forward Dynamic Programming algorithm, along with label correction, as follows:

Label Correcting DSP via DP

$OPEN \leftarrow \{s\}$, $g_s = 0$, $g_i = \infty \forall i \in Xn\{s\}$
while OPEN is not empty do
  Remove i from OPEN
  for $j \in Children(i)$ do
    if $(g_i + c_{ij}) < g_j$ and $(g_i + c_{ij}) < g_\tau$ then
      $g_j = g_i + c_{ij}$
      $Parent(j) = i$
      if $j \neq \tau$ then
        $OPEN = OPEN \cup \{j\}$

We use this label correcting algorithm to find an optimal policy for all our known maps.

However, for the unknown maps, the challenge was to compute a singular policy for one of 36 possible configurations. To solve this problem, we will build a policy for all possible configurations to compute the optimal action at any state. To do this, we first pre-compute a dictionary containing the list of all previous states given a state, thus finding all "neighbors" for the backward DP problem. This computation is very extensive, so is done once before computing the policy.

After computing this reverse state, we can now compute the policy from all potential goal states (which are encoded in our state space) and find the optimal actions leading to the goal state. We compute thi policy as follows:

Optimal policy calculation using backward DP

$goals = []$
for all goal + key + door configurations:
  for all directions + (has_key, not has_key)
    $goals \leftarrow (goal_pos, dir, has_key, door_state, key_pos, door_pos)$
$OPEN \leftarrow \{goals\}$, $g_{goals} = 0$, $g_i = \infty \forall i \in Xn\{s\}$
$Children(i)$ represents all states that can reach i with a single action.
Label correcting algorithm:
while OPEN is not empty do
  Remove i from $OPEN$
  for $j \in Children(i)$ do
    if $(g_i + c_{ij}) < g_j$ and $(g_i + c_{ij}) < g_\tau$ then
      $g_j = g_i + c_{ij}$
      $Parent(j) = i$
      $Policy(j) = action_{ji}$
      if $j \neq \tau$ then
        $OPEN = OPEN \cup \{j\}$

This global policy enables us to find the optimal action from any state to any of the defined goal states, with any door state and key state from the problem formulation.

IV. Results

A. Known Envs

We computed individual optimal policies for each known environment, since computing policies offline is faster online than computing the policy online. All the policies are optimal, as can be found by computing the value functions for each and checking whether they are minimized.

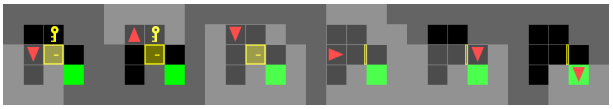


Fig. 1. Doorkey-5x5-normal

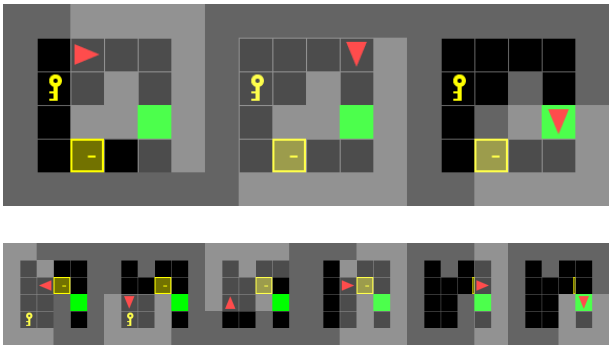


Fig. 2. Doorkey-6x6-direct and Doorkey-6x6-normal

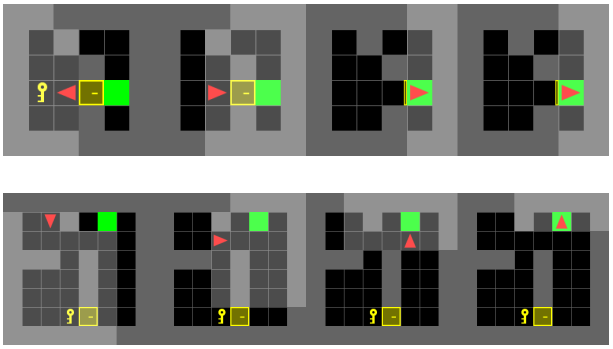


Fig. 3. Doorkey-6x6-shortcut and Doorkey-8x8-direct

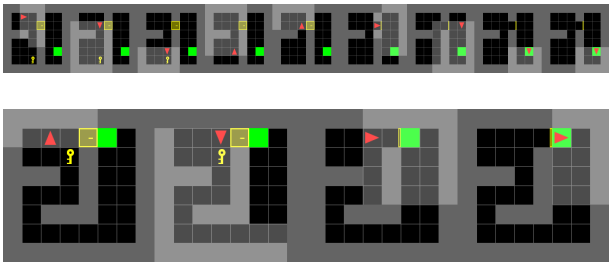


Fig. 4. Doorkey-8x8-normal and Doorkey-8x8-shortcut

B. Unknown Envs

We used the label correcting policy optimization to solve for the optimal control values and find our solution. This works remarkably well, finding solutions that are humanly unintuitive but are optimal to solving the problem.

An issue we ran into during implementation was underdefining the state space with only the spacial values of the robot. However, this resulted in the robot getting stuck facing walls due to a lack of consideration of the wall.

Another issue with implementation was attempting to update the environment as we moved through each state, which proved to be infeasible due to our inability to reset the environment to a prior state, which ties in with the underconstrained state space issue.

However, after solving all problems in the code, we managed to find a solution to all 36 random environments, which can be tested using the linked code, and can be seen from our solution images below. The images contain the last frame of the path taken by the robot to get to the goal, but we can obtain information about whether a door was opened or not due to the presence of the key, and which door was chosen by orientation of the robot in the last frame. We provided the gifs for the entire solution in our submitted code.

1) Goal position 1, Key position 1: We look at figures 5 and 6 for solution of environments with the first out of 3 random key and 3 random goal placements.

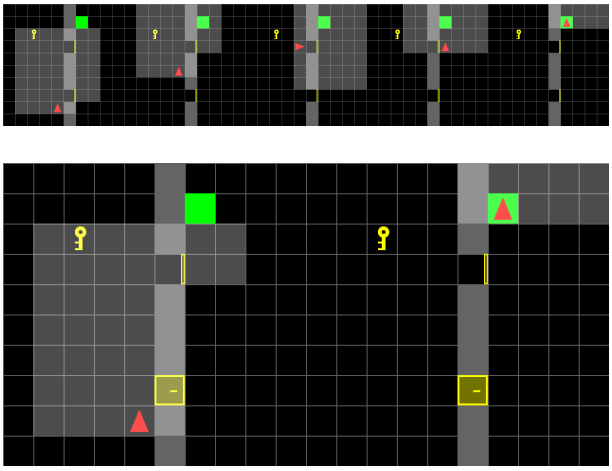


Fig. 5. Doorkey-10x10-1, both doors open and Doorkey-10x10-2, top door open

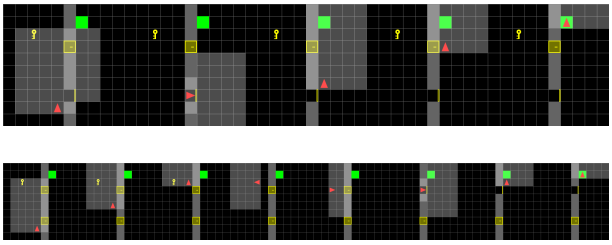


Fig. 6. Doorkey-10x10-3, bottom door open and Doorkey-10x10-4, no door open

2) Goal position 2, Key position 1: We look at figures 7 and 8 for solution of environments with the second out of 3 random key and 3 random goal placements.

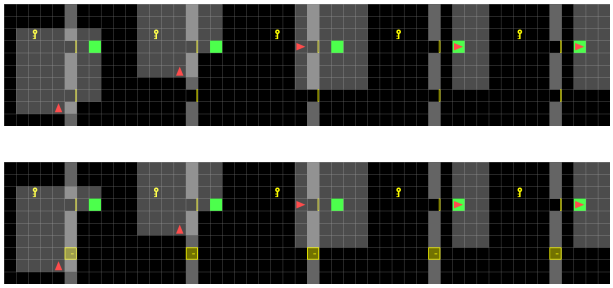


Fig. 7. Doorkey-10x10-5, both doors open and Doorkey-10x10-6, top door open

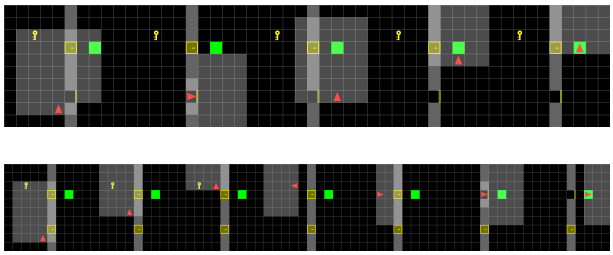


Fig. 8. Doorkey-10x10-7, bottom door open and Doorkey-10x10-8, no door open

3) Goal position 3, Key position 1: We look at figures 9 and 10 for solution of environments with the third out of 3 random key and 3 random goal placements.

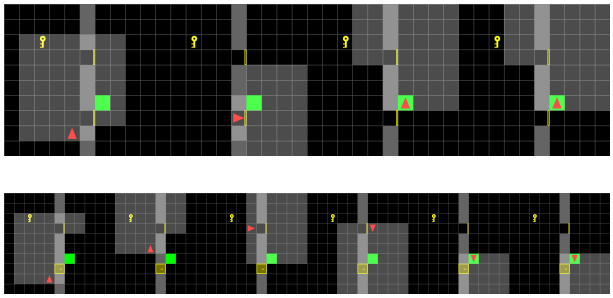


Fig. 9. Doorkey-10x10-9, both doors open and Doorkey-10x10-10, top door open

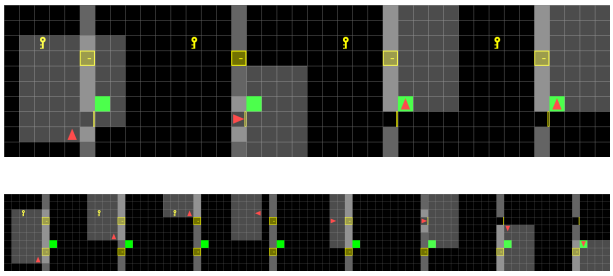


Fig. 10. Doorkey-10x10-11, bottom door open and Doorkey-10x10-12, no door open

4) Goal position 1, Key position 2: We look at figures 11 and 12 for solution of environments with the fourth out of 3 random key and 3 random goal placements.

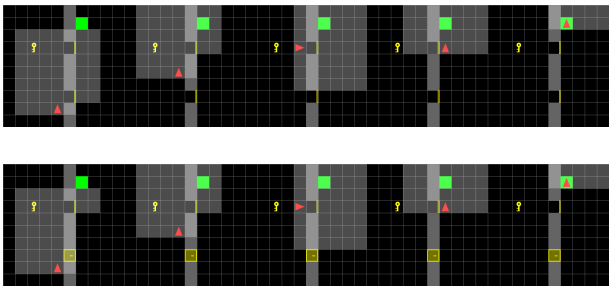


Fig. 11. Doorkey-10x10-13, both doors open and Doorkey-10x10-14, top door open

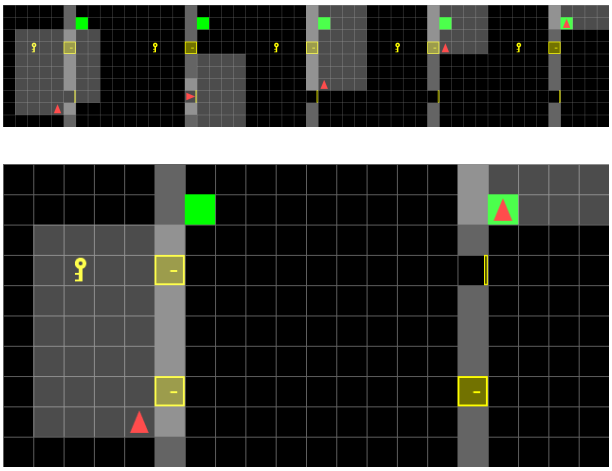


Fig. 12. Doorkey-10x10-15, bottom door open and Doorkey-10x10-16, no door open

5) Goal position 2, Key position 2: We look at figures 13 and 14 for solution of environments with the fifth out of 3 random key and 3 random goal placements.

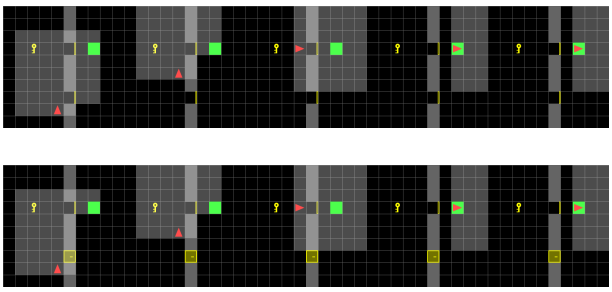


Fig. 13. Doorkey-10x10-17, both doors open and Doorkey-10x10-18, top door open

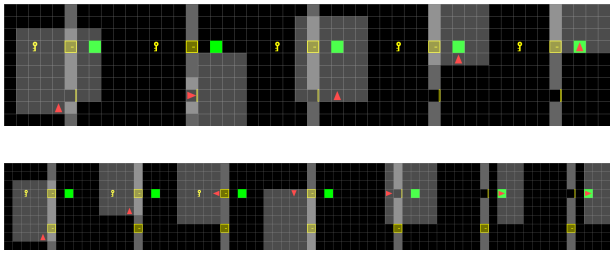


Fig. 14. Doorkey-10x10-19, bottom door open and Doorkey-10x10-20, no door open

6) Goal position 3, Key position 2: We look at figures 15 and 16 for solution of environments with the sixth out of 3 random key and 3 random goal placements.

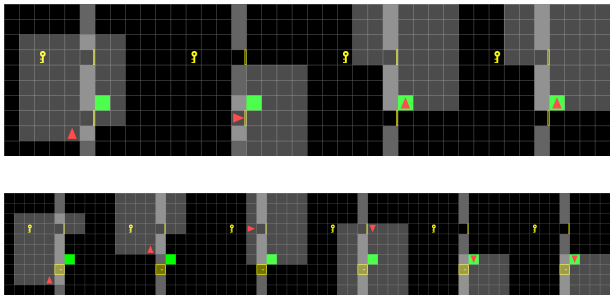


Fig. 15. Doorkey-10x10-21, both doors open and Doorkey-10x10-22, top door open

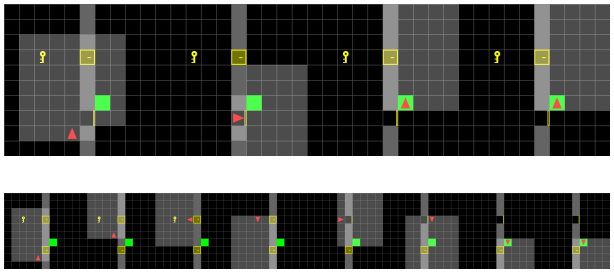


Fig. 16. Doorkey-10x10-23, bottom door open and Doorkey-10x10-24, no door open

7) Goal position 1, Key position 3: We look at figures 17 and 18 for solution of environments with the seventh out of 3 random key and 3 random goal placements.

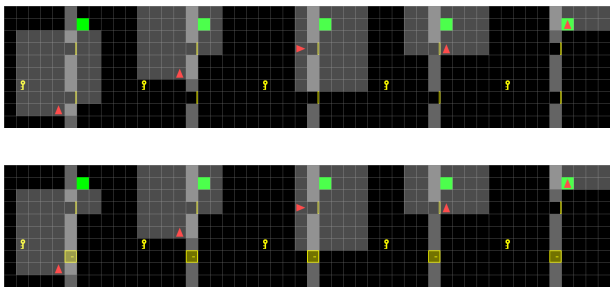


Fig. 17. Doorkey-10x10-25, both doors open and Doorkey-10x10-26, top door open

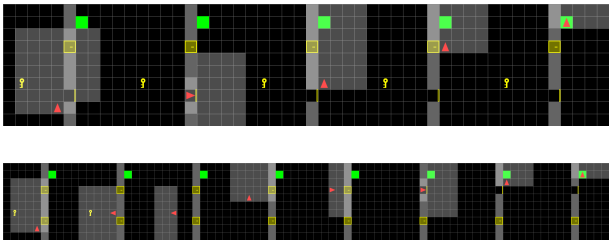


Fig. 18. Doorkey-10x10-27, bottom door open and Doorkey-10x10-28, no door open

8) Goal position 2, Key position 3: We look at figures 19 and 20 for solution of environments with the eighth out of 3 random key and 3 random goal placements.

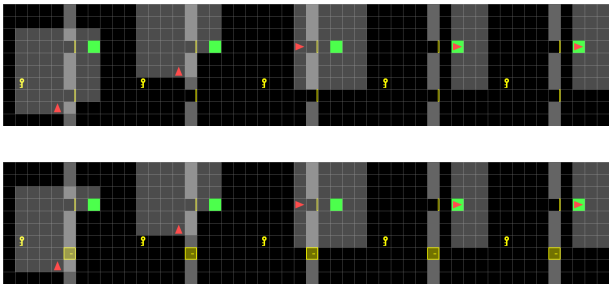


Fig. 19. Doorkey-10x10-29, both doors open and Doorkey-10x10-30, top door open

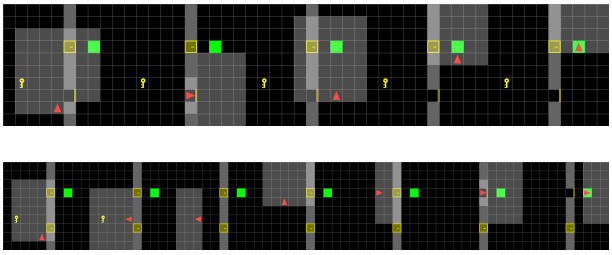


Fig. 20. Doorkey-10x10-31, bottom door open and Doorkey-10x10-32, no door open

9) Goal position 3, Key position 3: We look at figures 21 and 22 for solution of environments with the last out of 3 random key and 3 random goal placements.

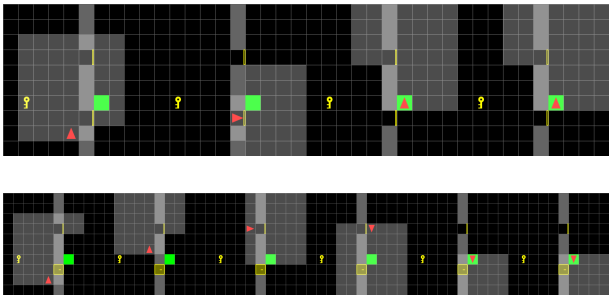


Fig. 21. Doorkey-10x10-33, both doors open and Doorkey-10x10-34, top door open

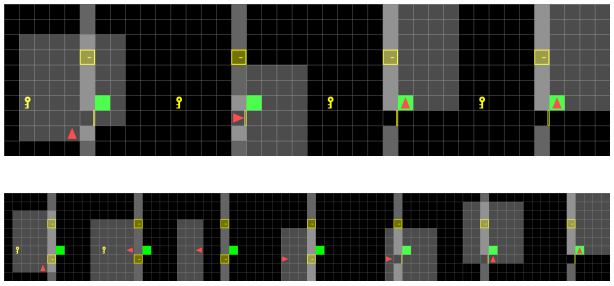


Fig. 22. Doorkey-10x10-35, bottom door open and Doorkey-10x10-36, no door open

V. Conclusion

In this report, we implemented a label correcting algorithm to solve the optimal control problem for a small, 2-dimensional environment with walls and locked doors. We formulated the problem as a Markov Decision Process, and then used that formulation to implement a Label Correcting Deterministic Shortest Path via Dynamic Programming.

Our results show the effectiveness of our work, and we can experimentally prove the optimality of our solutions. Future work will involve using improved planning algorithms like A^*, and further performance improvements in implementation for more complex, real-world problems.

References

1) Used Prof Atanasov's class notes and links provided in the project description.
2) Piazza discussion posts.
3) Discussed part B policy at a high level with Thomas.